Research Report Generator — Multi-Step LLM Agent

1. Problem Statement

A single LLM prompt asking "write me a research report on X" produces output constrained entirely by the model's training data. It cannot retrieve current information, cannot check its own reasoning, and has no mechanism to separate the act of gathering evidence from the act of synthesising it. These are fundamentally different cognitive operations and conflating them into one pass degrades quality at every stage.

The Research Report Generator solves this by decomposing the task into six sequential steps. The first step identifies what to look for. The second step actually looks. The third step structures what was found. The fourth step synthesises a coherent narrative. The fifth step critiques that narrative. The sixth step repairs the critique's findings. No single step can be removed without breaking the chain: the web search has nothing to search for without Step 1's sub-questions; the synthesis has no evidence without Step 3's key facts; the final revision has nothing to improve without the Step 5 critique. This dependency structure is what makes multi-step chaining genuinely valuable here rather than cosmetically applied.

2. Chain Design

Step 1 — Decompose receives only the user's raw topic string. Its sole job is to identify 3–4 specific, searchable sub-questions that together cover the topic. The output is a JSON object. This step exists separately because decomposition requires a different cognitive posture than writing: it is abstract, structural, and prior to any evidence. Folding it into the synthesis step would bias the questions toward what the model already knows.

Step 2 — Web Search (Tool) receives the list of sub-questions and issues a DuckDuckGo query for each one. It returns a dictionary mapping each sub-question to a list of title, snippet, and URL objects. This step is a tool call not an LLM call because the value it adds is retrieval, not reasoning. Asking an LLM to invent search results would be asking it to hallucinate.

Step 3 — Extract Facts receives the raw search snippets and returns a structured JSON list of key facts, each tagged with the sub-question it answers and a source URL. This step exists because raw HTML snippets are noisy, redundant, and unorganised. Passing them directly to a synthesis step would overwhelm the context window with irrelevant text. The extraction step acts as a filter, distilling signal from noise before synthesis begins.

Step 4 — Synthesise receives the structured key facts and produces a Markdown draft report. Only at this point does the agent write prose. Separating synthesis from extraction ensures the writer has clean, structured inputs rather than raw search debris.

Step 5 — Critique receives both the draft and the original key facts. It produces a JSON critique object identifying weaknesses, unsupported claims, and missing topics. This step exists because a single-pass writer cannot objectively evaluate its own output. The critique step adopts a different role adversarial reviewer and produces structured feedback that the final step can act on systematically.

Step 6 — Revise receives the draft and the critique and produces the final polished Markdown report. It addresses every weakness identified in Step 5 and adds a References section. The separation from Step 4 is essential: without the critique, revision is just rewriting; with it, revision is targeted repair.

3. Tool Integration

DuckDuckGo Search (DuckDuckGo-search library) was chosen as the external tool because it requires no API key, imposes no cost, and returns structured results consisting of title, snippet, and URL. It retrieves information the LLM cannot produce from training: recent events, specific statistics, and source attributions that give the final report verifiability rather than the false confidence of LLM confabulation.

The tool's output enters the chain as a Python dictionary keyed by sub-question. Step 3 receives this directly as part of its user prompt, structured so that the LLM can associate each snippet with the sub-question it was retrieved for.

4. Limitations

The chain fails when DuckDuckGo returns no results this happens for highly niche or recent topics, or when the search service rate-limits the client. The agent handles this gracefully by recording an empty list and continuing, but the downstream synthesis will note the gap. A production system would retry with alternate queries or fall back to a paid search API.

Step 3's extraction is bounded by the context window: if the topic is broad and returns many long snippets, the LLM may truncate or miss facts. The current MAX_SEARCH_RESULTS = 4 constant was chosen conservatively to avoid this, but it can be tuned.

The critique in Step 5 is only as good as the facts provided in Step 3. If Step 3 missed important information, Step 5 cannot identify it as missing it can only critique what it can see.

Finally, grok-3-mini occasionally produces JSON with extra commentary before or after the object. The parse_json_block() helpers handle this by scanning for the outermost curly braces or square brackets rather than parsing the full response, but truly malformed output can still cause a ValueError. A retry loop would make this robust in production.

5. Reflection

With more time, I would add a Step 2.5 that scores retrieved snippets for relevance before passing them to Step 3, discarding low-quality results. I would also add a retry loop with exponential backoff around the LLM calls and tool calls and stream the final report to the terminal rather than waiting for the entire chain to complete. The most non-obvious thing I learned was how much the output quality of Step 4 depends on Step 3: when the extraction prompt was vague, the synthesis received muddled facts, and the draft was correspondingly weak. Tightening the extraction JSON schema requiring a sub question field on every fact was the single change that most improved the final output.